

Wispr Flow for Numera — STT Fit Assessment

Wispr Flow for Numera — STT Fit Assessment

Date: 2026-07-21 **Question:** Should Numera use Wispr Flow for the voice tutor’s speech-to-text? **Short answer:** Wispr Flow is a **drop-in accuracy upgrade for the transcription stage**, and its batch/session model fits the half-duplex turn-based contract we just built surprisingly well. It is **not** a real-time voice platform — it does no turn detection, TTS, barge-in, or live captions, and access is gated. Treat it as an STT engine choice, not a voice-architecture decision.

1. What Wispr Flow’s API actually is

Verified against Wispr’s own developer docs (July 2026):

- **Batch REST, not streaming.** You POST a recorded utterance and get a transcript back. There is no documented WebSocket / live-streaming mode. This is the single most important fact for our architecture.
 - Returns **raw transcript + per-segment confidence scores**, with AI cleanup (filler removal, formatting) and server-side context-aware corrections.
 - **Persistent sessions** that carry context across calls within a work session.
 - Audio in as **base64 16 kHz PCM wav** (browsers record webm, so conversion is needed — but we already downsample to 16 kHz PCM16 in `useVoiceStream`, so we are ~90% there).
 - **No VAD / endpointing, no TTS, no barge-in** — it is purely the transcription stage.
 - **Access is gated** — enterprise approval required (email enterprise@wisprflow.ai); not instantly available.
 - Pricing: free tier 60 min/month; \$12/month for 600 min + API access. Volume for a multi-student product needs enterprise pricing.
-

2. Why it fits the half-duplex contract

Key insight: **because we committed to half-duplex + turn-based (the voice contract), Wispr’s batch model fits well.** In a barge-in / streaming world, batch STT would be a dealbreaker. But our flow is:

mic open during LISTENING → 2 s VAD decides the turn ended → *then* we have a complete utterance.

That is exactly **one clean audio clip per turn** — an ideal input for a batch transcriber. The contract’s design and Wispr’s API shape line up.

3. Concrete wiring in the Numera codebase

1. **Keep** useVoiceTurn’s VAD (the 2 s silence detector). Wispr does no turn detection, so this stays unchanged.
2. **Replace** the Web Speech API *final* result: record the turn’s PCM (reuse the existing `downsample / floatToPcm16` in `useVoiceStream`), wrap as wav + base64, POST to Wispr on turn-end.
3. Feed Wispr’s `transcript` → `voice_transcript` and its confidence → `transcript_confidence` in the `/interaction` request — **exactly the contract §5 fields already added**.
4. Low confidence → route straight into the contract’s **CLARIFICATION_REQUIRED** path (§14). Wispr’s per-segment confidence makes that gate real instead of guessed.
5. Use Wispr’s session id for the conversation continuity the contract’s history layer wants.

4. Tradeoffs vs the other options

	Web Speech (current rest)	Deepgram (current server)	Wispr Flow	STS (Realtime / Live)
Streaming / interim caption	Yes (live)	Yes (live)	No — batch,	Yes (native)
Accuracy + cleanup	Weak, flaky	Good	Best + AI cleanup	Good
Confidence scores	Partial	Yes	Yes, per-segment	n/a
Turn detection built-in	No	Yes	No (we keep VAD)	Yes
Fits half-duplex contract	OK	OK	Natural	Overkill
Latency after speaking	~instant	Low	+1 round-trip	Lowest
Availability	Free / now	Now	Gated approval	Now

5. The two real costs for Numera

1. **Loss of the live word-by-word caption** (audit Issue 3 — the single evolving text box), unless we keep Web Speech running *display-only* for interim while Wispr provides the authoritative final. Under half-duplex a brief “transcribing...” after the student stops is acceptable, but it is a small UX regression from a growing caption.
2. **Children’s voice audio → a third-party subprocessor** doing server-side corrections. Numera already gates on `voice_processing` consent, but Wispr becomes another data processor on kids’ speech — an NDA / privacy review item, not just an engineering one.

6. Recommendation

- Wispr Flow is the **best-accuracy option for the transcription stage specifically**, and it lines up cleanly with the half-duplex contract. It would materially help the “dropped / garbled sentence” complaints and gives a *real* confidence signal for §14.
- It is **not** a real-time voice engine: no turn-taking, TTS, barge-in, or live captions, and access is gated.

- Ranking for the current committed direction (half-duplex, turn-based):
 - **Deepgram** if we want to keep streaming live captions.
 - **Wispr Flow** if we want maximum accuracy + cleanup and can accept a post-utterance beat.
- If the real pain is *conversation feel* (latency, interruption, integration) rather than transcription quality, that is the **speech-to-speech** conversation (OpenAI Realtime / Gemini Live), not Wispr.

Either way, the turn-sync contract we built sits unchanged on top of whichever STT engine we pick.

Sources

- Flow for Developers — Wispr Flow: <https://wisprflow.ai/developers>
- Quickstart, Voice Interface API Documentation — Wispr Flow: <https://api-docs.wisprflow.ai/quickstart>
- WisprFlow for Developers and AI Builders (Zack Proser): <https://zackproser.com/blog/wisprflow-for-developers-ai>
- Wispr Flow: <https://wisprflow.ai/>